# PROJECT AND TEAM INFORMATION

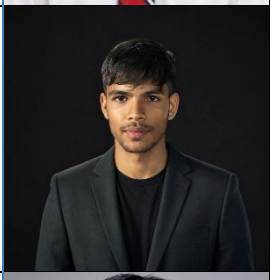
## Project Title

*ShelfSense: Intelligent Library Resource and Reservation System*

## Team ID : DSCPP-III-2026-T306

Course Name : Btech(CSE)

## Student / Team Information

| *Team* | *Team Name:* | *photo* |
|---|---|---|
| **Team member 1 (Team Lead)** *(name: student ID: email,picture):* | **Sarthak Pandey-2510380166** *2510380166@geu.ac.in university roll no-2028487 phone no-7897680580* |  |
| **Team member 2** *(name: student ID: email, picture):* | **Anurag Chauhan-2510370177** *2510370177@geu.ac.in University Roll No. -2028652 Phone no-6398498642* |  |
| **Team member 3** *( name: student ID: email, picture):* | **Ashutosh Singh -2510379890** *2510379890@geu.ac.in University roll no-2028675 Phone no- 7366001531* |  |
| **Team member 4** *(name: student ID: email, picture):* | **Pankaj kumar -2512510018** *2512510018@geu.ac.in university roll no – 2028789 phone no - 6205007902* |  |

# PROPOSAL DESCRIPTION

## Motivation

*Academic and institutional libraries manage thousands of physical books, diverse member profiles, dynamic circulation lifecycles, and high-demand reservations. In traditional setups, tracking book availability, monitoring borrower return deadlines, and managing waitlists are performed through manual logbooks or fragmented spreadsheets. This operational friction leads to unreturned books, unnoticed overdue penalties, missed reservation queues, and labor-intensive manual record verification.*

*The underlying challenge is fundamentally a data organization and scheduling problem: maintaining dynamic member and catalogue registries, managing fair allocation for high-demand items, tracking chronological return deadlines, and preserving a secure audit trail. These problems are solved efficiently through foundational Data Structures and Algorithms (DSA). ShelfSense is an automated, high-performance library management system built in C/C++. When a student issues a book, the system records the transaction and associates an immutable return deadline. The system continuously tracks active deadlines, generating proactive return reminders as deadlines approach and flagging overdue records once exceeded. By pairing core C data structures (Linked Lists, Queues, Min-Heaps, and Stacks) with C++ Object-Oriented Programming, ShelfSense eliminates manual bookkeeping, enforces reservation fairness, and automates deadline tracking without relying on cloud or heavy database infrastructure.*

## State of the Art / Current solution

*Current library administration across many educational institutions relies on manual physical registers, standard spreadsheet applications, or basic monolithic management software. While functional for baseline inventory entry, these approaches exhibit critical operational bottlenecks:*

- ***Manual Due-Date Tracking:*** *Librarians must periodically scan full member registries line-by-line to identify overdue loans, leading to human error and missed return deadlines.*
- ***Arbitrary Reservation Handling:*** *Waitlists maintained via spreadsheets or paper notes lack strict First-In, First-Out (FIFO) enforcement, causing priority conflicts when popular titles are returned.*
- ***Static Search & Memory Inefficiencies:*** *Basic systems rely on linear scans over unsorted flat tables, scaling poorly as book and transaction volumes increase.*
-

*ShelfSense addresses these systemic limitations through dedicated algorithmic structures:*

- ***Singly/Doubly Linked Lists:*** *Enable dynamic insertion, modification, and deletion of book and member records with zero fixed-memory waste.*
- ***FIFO Queue:*** *Enforces strict reservation priority for borrowed books.*
- ***Priority Queue (Min-Heap):*** *Maintains issue records sorted chronologically by due date (O(1) extraction of earliest deadlines) for automatic upcoming and overdue notifications.*
- ***Stack:*** *Maintains an undoable transaction and circulation history.*
- ***Binary Search & Sorting Algorithms:*** *Accelerate title, author, and member lookups.*
- ***Persistent File I/O:*** *Ensures robust data retention across system restarts.*

## Project Goals and Milestones

***General Goals:*** *Build a console-driven, robust C/C++ library management system that:*

- *Maintains complete book catalogues and student/member registries dynamically.*
- *Automates book issuing, returns, renewal cycles, and real-time availability tracking.*
- *Implements a priority-based due-date engine that extracts upcoming deadlines and alerts overdue accounts.*
- *Manages hold requests via a FIFO reservation queue when books are unavailable.*
- *Enables rapid searching (Binary/Linear) and sorting (Merge/Quick sort) across catalogues and member directories.*
- *Maintains an auditable transaction history using a stack-based structure.*
- *Persists all books, members, active issues, and reservations using local file handling (.txt/.dat).*
- *Demonstrates practical efficiency, encapsulation, and algorithm complexity in C/C++.*

***Milestones:***

- ***M1 (Weeks 1–2): Requirement Analysis & C Data Structures:*** *Finalize system schema; implement dynamic Linked Lists for Book and Member registries; develop initial console interface and CRUD operations.*

- ***M2 (Week 3): Reservation Queue & Waitlist Engine:*** *Implement FIFO standard Queue for book reservation handling; manage automatic allocation triggers when reserved items are returned.*

- ***M3 (Week 4): Priority Queue & Due-Date Tracking:*** *Construct a Binary Min-Heap (Priority Queue) keyed on due date timestamps; implement automated deadline scanning for pending return reminders and overdue alerts.*

- ***M4 (Week 5): C++ OOP Layer & Class Integration:*** *Encapsulate core logic into structured C++ classes (Book, Member, Library, Reservation, IssueTransaction, Notification); apply constructors, destructors, and access modifiers.*

- ***M5 (Week 6): Searching, Sorting & Transaction Stack:*** *Integrate Merge/Quick Sort for catalogue ordering; implement Binary Search for fast indexing; construct a Stack to record chronological issue/return audit logs.*

- ***M6 (Week 7): File Persistence & Notification Subsystem:*** *Implement file input/output streams for automated database saving/loading; build formatted console notification dashboards for overdue/due reminders.*

- ***M7 (Week 8): System Testing, Verification & Final Report:*** *Execute stress testing with comprehensive library datasets; benchmark algorithm time complexities; finalize project documentation and demonstration suite.*

## Project Approach

*ShelfSense is structured as a two-layer hybrid software architecture combining performance-oriented C data structures with an Object-Oriented C++ management controller.*

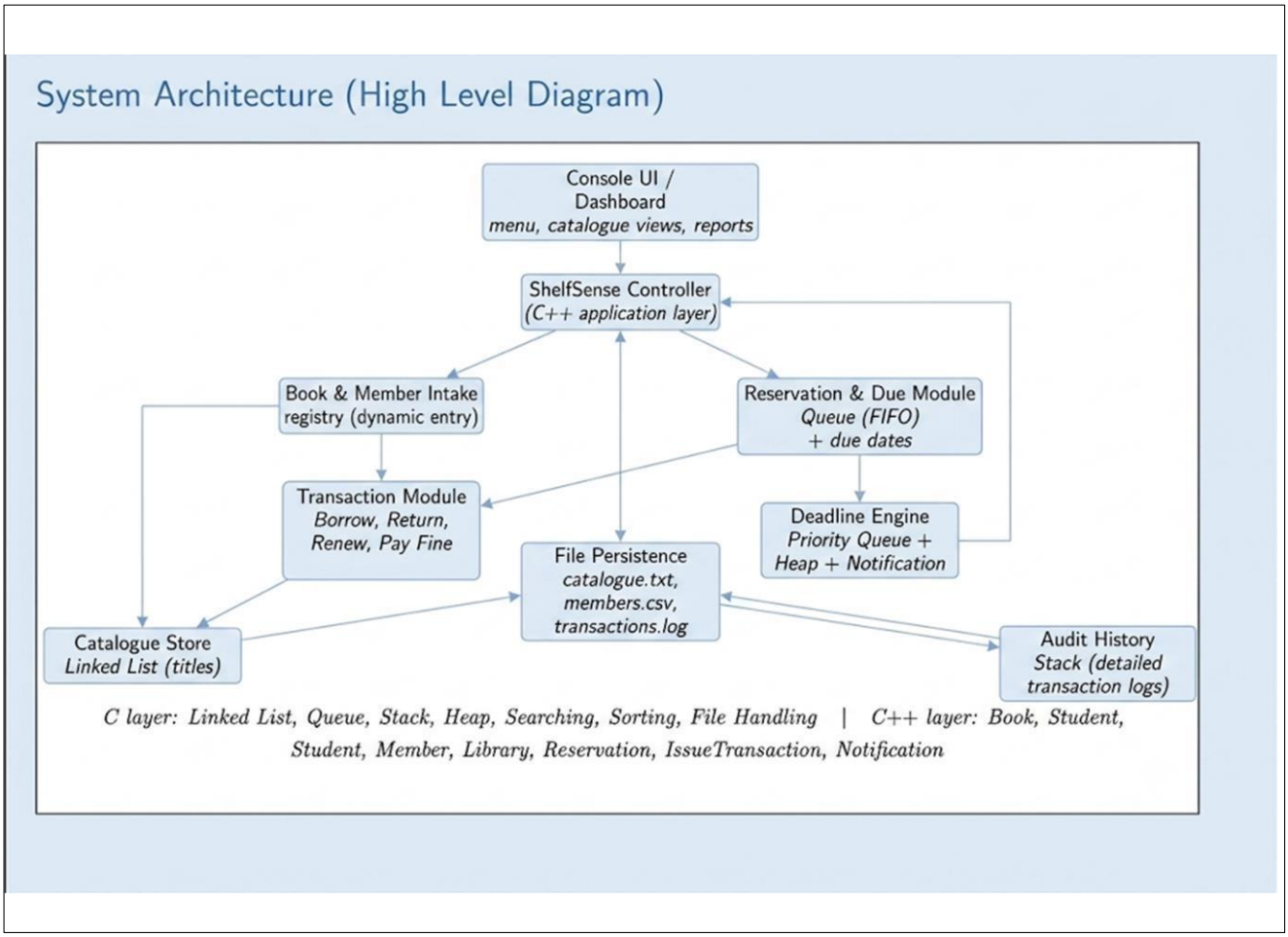
***C / Data Structures & Algorithms Layer:***

- ***Linked Lists:*** *Dynamic linear collections store book catalogues and member registries, enabling $O(1)$ head insertion and flexible node deletion without pre-allocating fixed memory.*
- ***Reservation Queue:*** *A standard FIFO queue manages waitlists for checked-out books, ensuring that returned copies are automatically reserved for the earliest requester.*
- ***Min-Heap (Priority Queue):*** *Organizes active book issues using return due dates as keys. The earliest deadline is always accessible at the root ($O(1)$ lookup, $O(\log n)$ insertion/deletion), driving the deadline notification and overdue tracking engine.*
- ***Transaction Stack:*** *Records issue and return events sequentially ($O(1)$ push), providing a reversible audit history of recent desk transactions.*
- ***Search & Sort Routines:*** *Implements Quick/Merge Sort for alphabetizing or categorizing inventories, alongside Binary Search ($O(\log n)$) for record retrieval.*
- ***File Storage:*** *Formatted file streams (fstream) handle read/write serialization to maintain persistent records across executions.*

***C++ OOP Layer:***

- ***Class Encapsulation:*** *Dedicated classes (Book, Student/Member, Library, Reservation, IssueTransaction, Notification) isolate internal data members behind controlled public interfaces.*
- ***Resource Management:*** *Constructors and destructors manage dynamic pointers and heap allocations safely, preventing memory leaks.*
- ***Modularity:*** *Decouples core algorithmic operations from UI and file management routines, keeping the codebase maintainable and extensible.*

## System Architecture (High Level Diagram)




## Project Outcome / Deliverables

- *A fully functional, modular C/C++ library management application compiled via GCC/G++.*
- *Dynamic management of book inventories and member directories utilizing Linked Lists.*
- *An automated issue and return tracking engine with real-time book status updates.*
- *A FIFO Queue reservation system for fair and structured allocation of high-demand titles.*
- *A Binary Min-Heap Priority Queue for due-date tracking, enabling immediate lookup of earliest return deadlines.*
- *Automated notification dispatch for upcoming due-date reminders and flagged overdue accounts.*
- *High-performance Binary Search ($O(\log n)$) and Quick/Merge Sort routines for library record organization.*
- *An auditable, stack-based transaction history log for circulation tracking.*
- *Persistent local file storage for seamless data loading and saving between sessions.*

## Assumptions

- The system is an offline, single-user, console-based C/C++ application running in a local environment.
- All operational data is either imported via local configuration/data files or input via standard console entry.
- Every issued book is assigned a fixed lending duration to calculate a definitive due date.
- System date/time functions are used to evaluate deadlines and identify overdue thresholds.
- Book reservations operate strictly under FIFO constraints per specific title.
- Notifications and alerts are generated internally and displayed directly within the console dashboard without third-party email/SMS integrations.

## References


- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., *Introduction to Algorithms*, 4th ed., MIT Press (Priority Queues, Heaps, Searching & Sorting).
- Stroustrup, B., *The C++ Programming Language*, 4th ed., Addison-Wesley (Object-Oriented Design, Classes, Encapsulation & Memory Management).
- Kernighan, B. W., & Ritchie, D. M., *The C Programming Language*, 2nd ed., Prentice Hall (Pointers, Dynamic Structures & File I/O).
- Drozdek, A., *Data Structures and Algorithms in C++*, 4th ed., Cengage Learning (Linked Lists, Queues, Stacks & Heap Structures).
- Sedgewick, R., & Wayne, K., *Algorithms*, 4th ed., Addison-Wesley (Sorting, Binary Search & Priority Queue Applications).